SOFTOMATIC
TECHNOLOGY SERVICES

EXAMATE · GENAI + PROJECTS · SESSION 6 · THU 29 OCT

GenAI Session 6
Docker and Docker Compose

Today your project stops depending on your laptop. One command starts your API and a Postgres database with pgvector, the same way on every machine.

SESSION
90 minutes
12:45 Group A + T · repeated 2:15 Group B + T

YOU LEAVE WITH
docker compose up
API + Postgres/pgvector + /ready

NEXT · FRI 30 OCT
CI and going live
tag week-2

TODAY'S JOURNEY

01 Why containers
02 Read the Dockerfile
03 Compose: API + Postgres
04 Add /ready
05 **Secrets done right**

SESSION 6 · GENAI + PROJECTS

One command, every laptop

Until now your API ran only where you installed everything by hand. Today you package it in a **container** and start it next to a real **Postgres + pgvector** database, which Week 3 needs for search.

By the end of today you can

01

Explain image vs container

and why "works on my machine" stops being a problem

02

Read a Dockerfile

line by line, for your FastAPI app

03

Run API + Postgres

with pgvector, using one docker compose command

04

Handle secrets

.env on your laptop, never in Git or the image

The 90-minute plan

0–10	Recap + hook	Session 5 in 3 questions; "it works on my machine"
10–25	Images and containers	the meal-kit picture; read the Dockerfile
25–40	Compose + secrets	one network, names not localhost, volumes, health checks
40–75	Build	docker-compose.yml, /ready, pgvector check
75–85	Break it on purpose	stop the database: /health stays 200, /ready goes 503
85–90	Checkpoint	show the trainer; tonight's task

Bring

Your repo with /chat from Sessions 4–5 · Docker Desktop installed and running · About 2 GB free disk · Laptop charger

No Docker? See section 11

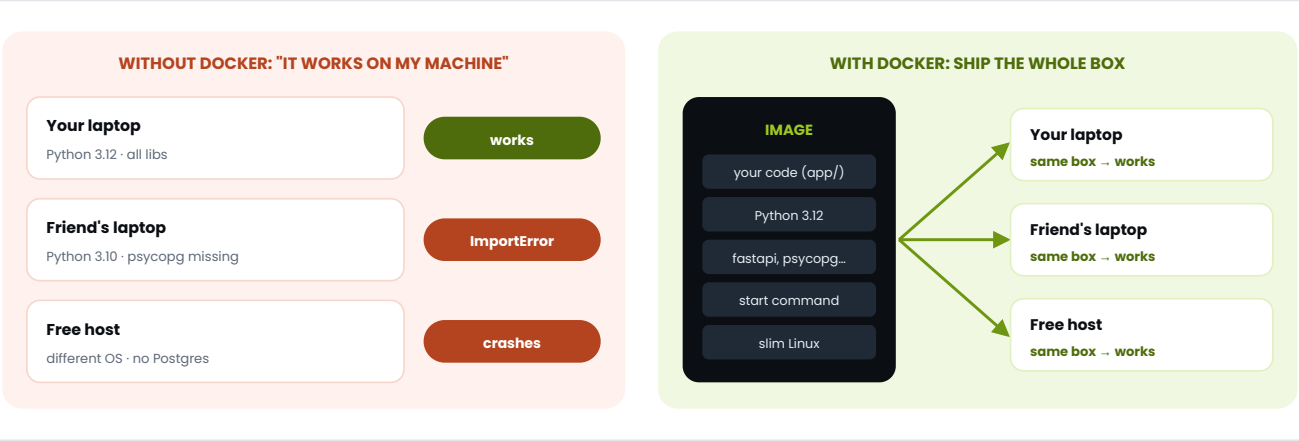
MEMORY HOOK

Ship the box, not the instructions. A container carries your code, Python and every library together, so it runs the same everywhere.

WHY THIS MATTERS

"It works on my machine"

Your friend clones your repo to help you debug. Twenty minutes later they are still installing the right Python and a Postgres server. In a company, every new teammate, the CI server and the production host face the same problem. Docker solves it by shipping the whole box.



Left: every machine is set up by hand, so each one is different. Right: one image carries everything, so every machine runs the same thing.

Just for fun

"It works on my machine" is not a deployment strategy. It is a confession.

Why today, in Week 2

Week 3 stores your chunks and embeddings in Postgres with pgvector. Tomorrow you deploy the same image to a free host. Both need the container you build today.

PAUSE AND PREDICT

You run `docker compose down` and then `up` again. Is the data in your database still there?

Answer: yes. The database files live in the named volume **pgdata**, which survives. Only `docker compose down -v` deletes volumes, and the data with them.

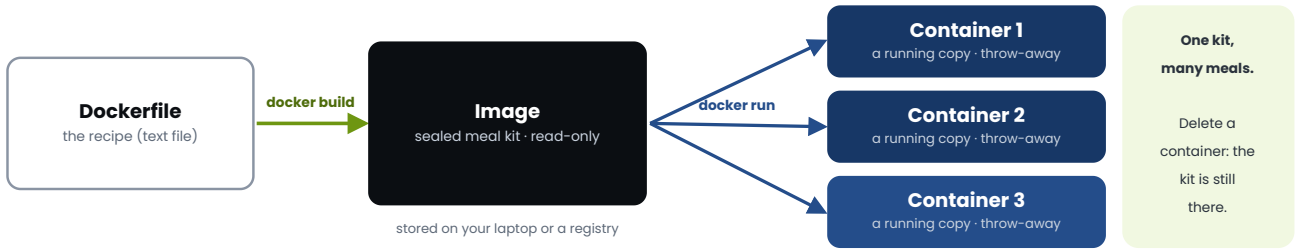
INTERVIEW MODE

"My API and its Postgres database run as containers started together with Docker Compose, so any machine with Docker runs exactly what I run."

ADDED FOR THIS SESSION

Image and container, in one picture

Two words cause most of the confusion. An **image** is a packed, read-only box: code, Python, libraries and the start command. A **container** is that image running. One image can run as many containers.



Dockerfile → `docker build` → image → `docker run` → containers. Compose runs several containers together from one YAML file.

WORD	PLAIN MEANING	IN OUR PROJECT
Dockerfile	The recipe: a text file of build steps	`Dockerfile` in your repo (already in the template)
Image	The sealed meal kit built from the recipe	`examate-rag`, built by `docker build`
Container	A running copy of an image; throw-away	the **api** and **db** containers
Registry	A shop for images	Docker Hub supplies `pgvector/pgvector:pg16`
Port mapping	A door from your laptop into a container	`"8000:8000"` = laptop port : container port
Container vs virtual machine	A VM carries a whole operating system; a container shares your machine's kernel	containers start in seconds and use far less memory

MEMORY HOOK

Recipe, kit, meal. Dockerfile is the recipe, the image is the kit, the container is the meal being cooked. Throw away the meal; the kit is still there.

SAY THESE CORRECTLY

Today's vocabulary

Seven terms from the session doc. CI, GitHub Actions and smoke tests are tomorrow's words.

TERM	MEANING
Docker Compose	A tool that starts several containers together from one YAML file, with networking between them.
Service (Compose)	One container definition in docker-compose.yml, such as api or db.
Volume	Storage that survives container restarts, used here for the database files.
Health check (container)	A command Docker runs to decide if a container is healthy.
pgvector	A Postgres extension that adds a vector column type and nearest-neighbour search.
Secret	A value that grants access (API key, database password). Must never be in Git or in an image.
Ephemeral filesystem	Disk that is wiped when the container restarts or redeploys.

Liveness vs readiness

/health (liveness) answers "is the process up?" and touches nothing. **/ready** (readiness) answers "can I serve real requests?" and checks the database. You build /ready today.

Why pgvector now?

In Week 3 every chunk of your documents gets an embedding (a list of numbers). pgvector stores them in Postgres and finds the closest ones. Today you only check it is installed.

COMMON MISTAKE

Saying "container" when you mean "image". Interviewers notice. You **build** an image; you **run** a container.

ADDED FOR THIS SESSION: THE SESSION DOC HAS NO DOCKERFILE WALKTHROUGH

Your Dockerfile, line by line

The template already has this file. You do not write it from scratch; you must be able to explain every line.

DOCKERFILE (FROM THE EXAMATE TEMPLATE)

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r
requirements.txt
COPY app ./app
RUN useradd --create-home appuser && mkdir -p
/app/logs && chown -R appuser /app
USER appuser
EXPOSE 8000
CMD ["sh", "-c", "uvicorn app.main:app --host
0.0.0.0 --port ${PORT:-8000}"]
```

.DOCKERIGNORE

```
.venv
.env
data/raw
.git
__pycache__
.pytest_cache
.ruff_cache
logs
```

LINE	WHY IT IS THERE
`FROM python:3.12-slim`	Start from a small official image that already has Python 3.12 on Linux.
`ENV PYTHONDONTWRITEBYTECODE=1 …`	No .pyc clutter; logs print immediately instead of being buffered.
`WORKDIR /app`	All later commands run inside /app in the container.
`COPY requirements.txt . + RUN pip install`	Libraries first, code later: Docker reuses this slow layer when only your code changes.
`COPY app ./app`	Your code. Only app/ is copied; .dockerignore keeps .env, .venv and data/raw out.
`RUN useradd … · USER appuser`	Run as a normal user, not root: a basic security habit.
`EXPOSE 8000`	Documents the port the app listens on.
`CMD … --port ${PORT:-8000}`	Start uvicorn on $PORT if the host sets one (tomorrow), else 8000.

Order matters: layers and the build cache



Each line makes a layer. When a layer changes, it and every layer above it are rebuilt. Code changes daily, so it goes last.

BEFORE COMPOSE

Build it, run it, call it

First prove the API image works on its own. Run these from your repo folder.

TERMINAL

```
# 1. build the image and name it
docker build -t examate-rag .

# 2. run one container; -p maps laptop port 8000 to container port 8000
docker run --rm -p 8000:8000 --env-file .env examate-rag

# 3. in a second terminal
curl localhost:8000/health

# 4. see what is running, then stop it with Ctrl+C in the first terminal
docker ps
```

First build is slow

It downloads Python and installs your libraries. The second build after a code change takes seconds, thanks to the cache.

--env-file .env

Settings are passed in when the container **starts**. They are never copied into the image.

Useful commands

`docker images` · `docker ps -a` · `docker logs NAME` · `docker system df` (disk used)

PAUSE AND PREDICT

You change one line in `app/main.py` and run `docker build` again. Does pip reinstall all your libraries?

Answer: no. requirements.txt did not change, so the pip layer comes from the cache. Only the `COPY app` layer and the lines after it are rebuilt.

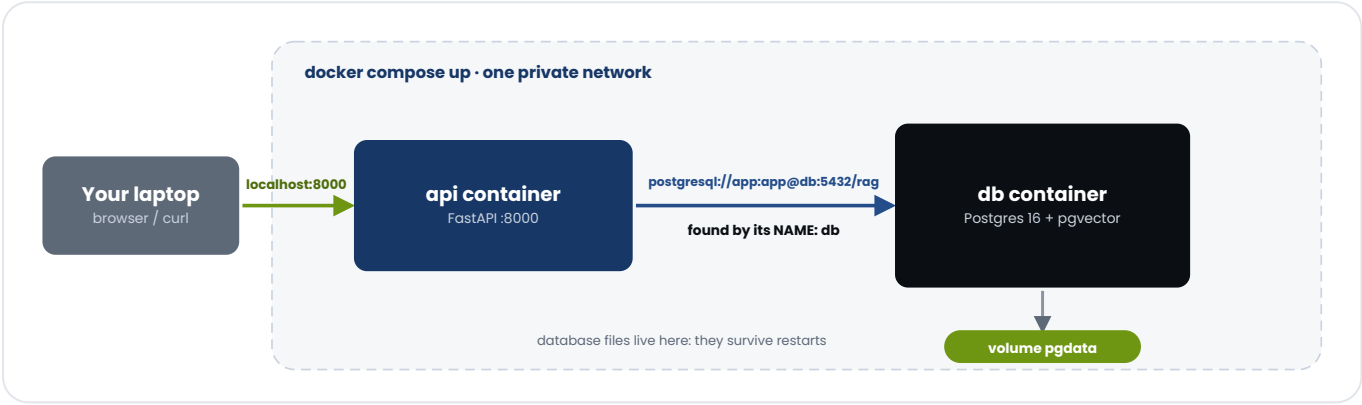
INTERVIEW MODE

"I copy requirements.txt and install dependencies before copying the code, so code changes rebuild in seconds. The image runs as a non-root user, and configuration comes from environment variables at run time."

API + DATABASE ON ONE NETWORK

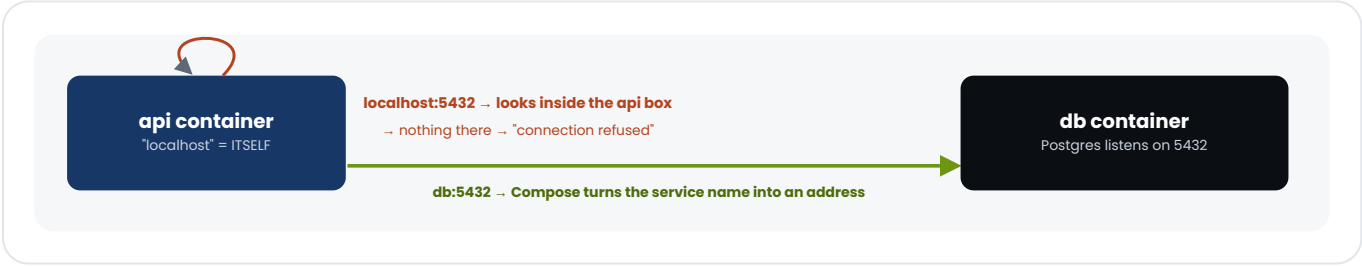
Containers talk by name

Compose starts both containers with one command and puts them on a private network. Inside that network each service is reachable by its name.



Inside Compose, containers reach each other by service name (db), not localhost; the volume keeps database files when containers are recreated.

The localhost trap



WHAT GOES WRONG

Writing `localhost:5432` in DATABASE_URL. Inside the api container, localhost is the api container itself, so the database is never found. Use `db:5432`.

MEMORY HOOK

Inside Compose, call people by name. localhost means "me".

BUILD · 20 MINUTES

Write docker-compose.yml

Create this file in the root of your repo, next to the Dockerfile.

DOCKER-COMPOSE.YML · TOP: THE DATABASE

```yaml
services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: app
      # local development only; never reuse in production
      POSTGRES_PASSWORD: app
      POSTGRES_DB: rag
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d rag"]
      interval: 5s
      timeout: 3s
      retries: 10
```

DOCKER-COMPOSE.YML · CONTINUED: THE API + VOLUMES

```yaml
  api:
    build: .
    env_file: .env
    environment:
      DATABASE_URL: postgresql://app:app@db:5432/rag
    ports:
      - "8000:8000"
    depends_on:
      db:
        condition: service_healthy

volumes:
  pgdata:
```

PART	WHAT IT DOES	PART	WHAT IT DOES
`pgvector/pgvector:pg16`	Postgres 16 + pgvector, from Docker Hub	`build: .`	Builds the api image from your Dockerfile
`POSTGRES_PASSWORD: app`	Local only; never reuse in production	`env_file: .env`	Your settings and keys, read at start-up
`ports "5432:5432"`	Tools on your laptop can reach the database (optional)	`DATABASE_URL … @db:5432`	Uses the service name **db**, not localhost
`volumes: pgdata`	Database files survive restarts	`depends_on … service_healthy`	The api waits until Postgres answers
`healthcheck`	`pg_isready` every 5 s, up to 10 tries	`volumes: pgdata:` (last line)	Declares the named volume



Docker checks every 5 s, up to 10 times (about 50 s), before calling db unhealthy. The api waits for "healthy".

ONE COMMAND

Start everything

Make sure your .env exists and contains `LLM_PROVIDER=fake` for now, so nothing calls a real model during setup.

TERMINAL 1

```
cp .env.example .env          # only if you have no .env yet
docker compose up --build
```

TERMINAL 2

```
curl localhost:8000/health
curl localhost:8000/ready     # add the route on the next page first

# check pgvector is available
docker compose exec db psql -U app -d rag -c "CREATE EXTENSION IF NOT EXISTS vector; SELECT extversion
FROM pg_extension WHERE extname = 'vector';"
```

Everyday Compose commands

COMMAND	WHAT IT DOES
`docker compose up --build`	Build the api image if needed and start both services (logs in this terminal)
`docker compose up -d`	Same, in the background
`docker compose ps`	Which services run, and whether db is healthy
`docker compose logs -f api`	Follow the api logs
`docker compose stop db`	Stop one service (used in the experiment in section 09)
`docker compose down`	Stop and remove containers; the pgdata volume stays
`docker compose down -v`	**Also deletes volumes: your database data is gone**

PAUSE AND PREDICT

Remove the `condition: service_healthy` line from the compose file. What can happen on the first start?

The api can start before Postgres is ready and fail its first database call.

COMMON MISTAKE

Running `down -v` "to clean up" in Week 3, after ingesting 500 pages. Use plain `down` unless you really want an empty database.

BUILD · 10 MINUTES

The readiness endpoint

Add this to `app/main.py`. It needs `database_url: str = ""` in Settings and `psycopg[binary]` in requirements.txt; both came in Session 4. If yours are missing, add them now.

APP/MAIN.PY · THE NEW PART

```python
import psycopg
from fastapi.responses import JSONResponse


@app.get("/ready")
def ready(settings: SettingsDep):
    """Readiness: dependencies are reachable. Load balancers and hosts use this."""
    if not settings.database_url:
        return {"status": "ready", "database": "not configured"}
    try:
        with psycopg.connect(settings.database_url, connect_timeout=3) as conn:
            conn.execute("SELECT 1")
        return {"status": "ready", "database": "ok"}
    except Exception as exc:  # report, don't crash
        return JSONResponse(status_code=503, content={"status": "not ready", "database": str(exc)[:200]})
```

TESTS/TEST_HEALTH.PY · ADD THIS TEST

```python
def test_ready_without_database(client):
    r = client.get("/ready")
    assert r.status_code == 200
    assert r.json()["database"] == "not configured"
```

Three answers /ready can give

- **200 · not configured**: no DATABASE_URL (tests, or running without Compose)
- **200 · ok**: `SELECT 1` worked
- **503 · not ready**: the database could not be reached; the error is reported, the app does not crash

MEMORY HOOK

Health says "I am alive". Ready says "I can work".

THE EXPERIMENT

Liveness vs readiness

With Compose running, call both endpoints. Then stop the database and call them again.



TERMINAL 2

```
curl -i localhost:8000/ready      # 200 {"status":"ready","database":"ok"}
docker compose stop db
curl -i localhost:8000/health     # still 200
curl -i localhost:8000/ready      # 503 {"status":"not ready", ...}
docker compose start db           # wait a few seconds, then /ready is 200 again
```

Who uses this difference

Load balancers and hosts send traffic only to instances that are **ready**, and restart instances that are not **alive**. You met load balancers in System Design L5.

Why /health must not check the database

If /health failed whenever the database blinked, the host would restart healthy API servers that cannot fix the database anyway.

INTERVIEW MODE

"My liveness check touches nothing. My readiness check runs SELECT 1 against Postgres with a 3-second timeout and returns 503 if it fails, so traffic only reaches instances that can serve it."

CONFIGURATION AND SECRETS IN THREE PLACES

Keys stay out of Git and out of the image

Settings that change between places (database URL, model name, API key) come from **environment variables**. Code stays the same everywhere.



WHERE	HOW SECRETS GET IN	WHAT MUST NEVER HAPPEN
Your laptop	.env file (git-ignored), loaded by pydantic-settings and by Compose env_file	Committing .env
CI (GitHub Actions)	Repository secrets, exposed as environment variables to steps that need them; our CI uses the fake provider, so it needs no secret at all	Printing secrets in logs
Host	The host's environment-variable / secret settings	Baking secrets into the Docker image with ENV or copying .env into it

Check yours in 30 seconds

- [ ] `git status` never lists .env
- [ ] .env is in both .gitignore and .dockerignore
- [ ] .env.example has the names, no real values

If a key ever reaches Git

Deleting the file is not enough: the key is in history. **Rotate the key at the provider** (create a new one, delete the old one), then fix .gitignore.

NEVER DO THIS

Put `ENV LLM_API_KEY=…` in the Dockerfile or `COPY .env` into the image. Anyone who can pull the image can read the key, and changing it needs a rebuild.

ADDED FOR THIS SESSION

Install notes and a no-Docker plan

Install Docker Desktop before class. If your laptop cannot run it, you still finish today's work; you just run it somewhere else.

Windows

Docker Desktop with the **WSL 2** backend. Docker's docs list: Windows 10 22H2 or Windows 11 23H2+, WSL 2.1.5 or later, **8 GB RAM**, and **virtualization enabled in BIOS/UEFI** [S2]. Run `wsl --update` first.

macOS

Download the Docker Desktop build for your chip: Apple silicon (M1/M2/M3…) or Intel. Open it once and wait for "Engine running".

Linux

Install Docker Engine and the Compose plugin from Docker's docs for your distribution. Check with `docker compose version`.

Laptop cannot run Docker? Pick one

OPTION	HOW	GOOD TO KNOW
1. GitHub Codespaces	On your repo page: Code → Codespaces → Create. A cloud machine opens in the browser with Docker inside; run the same commands there.	Free accounts get 120 compute hours a month, counted ×2 on a 2-core machine: about 60 hours [S3]. Stop it when done.
2. Pair with a neighbour	Write the files on your laptop and push; run `docker compose up` on your partner's laptop from your branch.	You still explain every line yourself.
3. Without Docker, for now	`uvicorn app.main:app --reload` with DATABASE_URL empty: /ready says "not configured" and the test passes.	Still commit the Dockerfile and compose file; tomorrow's CI builds the image for you.

COMMON WINDOWS PROBLEMS

"WSL 2 installation is incomplete" → run `wsl --update` as administrator and restart. "Virtualization must be enabled" → turn on Intel VT-x / AMD-V (SVM) in BIOS. "Cannot connect to the Docker daemon" → Docker Desktop is not running: open it and wait.

SHOW THE TRAINER

Today's checkpoints

From the session doc's checkpoint list, the Docker part. The rest is tomorrow.

Before you leave

- [ ] `docker compose up` → `/ready` returns `database: ok`
- [ ] Stopping db makes `/ready` return 503 while `/health` stays 200
- [ ] pgvector check prints a version number
- [ ] `pytest -q` passes, including `test_ready_without_database`
- [ ] `git status` does not show .env; Dockerfile and docker-compose.yml are committed and pushed

Next session · GenAI 7 · Fri 30 Oct: CI and going live

The second half of the same session doc: a GitHub Actions pipeline that lints, tests with the fake LLM, builds your image and smoke-tests /health on every push; pre-commit checks; deploying your image to a free host with secrets in its settings; the live URL and CI badge in your README; and the **week-2** tag. Today's Dockerfile and compose file are what that pipeline builds.

PAUSE AND PREDICT

Tomorrow the host runs your image without Compose and without a database. What will /ready return there?

200 with `"database": "not configured"`, because DATABASE_URL is empty on the host this week. That is expected.

FIX IT YOURSELF FIRST

Common errors and fixes

The first two rows are from the session doc; the rest are the ones we see most in class.

SYMPTOM	CAUSE	FIX
API in Compose cannot reach localhost:5432	Inside Compose, localhost is the API container itself	Use the service name: db:5432 (already set in DATABASE_URL).
port is already allocated for 5432	A local Postgres is running	Stop it, or change the left side of the port mapping, e.g. "5433:5432".
port is already allocated for 8000	An old uvicorn or container still uses 8000	Stop it (Ctrl+C, `docker ps` then `docker stop NAME`), or map "8001:8000".
env file .env not found	No .env in the repo folder	`cp .env.example .env`, then set LLM_PROVIDER=fake.
/ready returns 503 right after start	Postgres still starting, or depends_on condition missing	Wait for "healthy" in `docker compose ps`; keep `condition: service_healthy`.
ModuleNotFoundError: psycopg in the container	Added to your laptop but not to requirements.txt	Add `psycopg[binary]` to requirements.txt; `docker compose up --build`.
Code change not visible	Old image still running	`docker compose up --build` rebuilds the api image.
Database empty after restart	`down -v` deleted the volume	Use plain `down`; re-ingest if needed.
Cannot connect to the Docker daemon	Docker Desktop is not running	Open Docker Desktop and wait for "Engine running".
Image pull very slow or times out	Classroom Wi-Fi	Pull once on a hotspot: `docker pull pgvector/pgvector:pg16`.

MEMORY HOOK

Read the last 20 lines. `docker compose logs api` almost always names the problem.

PRACTISE ALOUD WITH A PARTNER

Interview questions on containers

PATTERN-ONLY = a common question pattern in backend and GenAI interviews; no specific company is claimed. Q1 and Q2 are from the session doc; the CI and free-hosting questions come tomorrow.

Q1. How do you manage secrets for your application?

PATTERN-ONLY

> **Model answer.** "Secrets never go into Git or into the image. Locally they live in a git-ignored .env file; in CI they are repository secrets, and my tests don't need any because they use a fake LLM; in production they're set in the host's environment or secret manager and read at startup. If a key leaks, I rotate it immediately, because deleting it from the repo doesn't remove it from history."

Follow-up 1. Why not bake the key into the image? → Anyone who can pull the image can read it, and rotating requires a rebuild.

Follow-up 2. A teammate needs the key? → Share access to the secret store, not the key in chat; give per-person keys where the provider allows.

Follow-up 3. Many services and environments? → A central secret manager with access control, audit logs and automatic rotation.

Weak: "I keep it in config.py."

Strong: "Env-based config, git-ignored locally, secret store in CI and prod, rotate on leak."

Q2. Why Docker Compose, and how do containers find each other?

PATTERN-ONLY

> **Model answer.** "Compose starts the API and Postgres with one command, on a shared network where each service is reachable by its name, so the API connects to db:5432. A named volume keeps database data across restarts, and a health check makes the API wait until Postgres is ready."

Follow-up 1. Is Compose used in production? → Sometimes for small single-host deployments; larger systems use orchestrators like Kubernetes or managed platforms.

Follow-up 2. Database data disappears after docker compose down? → Only with down -v, which deletes volumes.

Follow-up 3. How would this map to production? → Managed Postgres with pgvector, the API as a container service with autoscaling, secrets from the platform.

Weak: "Compose is Docker for many things."

Strong: "One network, DNS by service name, volumes for state, health-gated startup."

ONE MORE, THEN REVISE

Question 3 and the 30-second answer

Q3. What is the difference between an image and a container? (added for this session) PATTERN-ONLY

> **Model answer.** "An image is a read-only package built from a Dockerfile: the code, the runtime, the libraries and the start command. A container is a running instance of that image with its own process and writable layer. I can start many containers from one image and throw them away without changing the image."

Follow-up 1. Why copy requirements.txt before the code? → So the dependency layer is cached and code changes rebuild in seconds.

Follow-up 2. Container vs virtual machine? → A VM runs a full guest operating system; containers share the host kernel, so they start faster and use less memory.

Weak: "They are the same thing."

Strong: "Image = the built package; container = a running instance of it."

YOUR 30-SECOND ANSWER: "HOW DO YOU RUN YOUR PROJECT?"

"It's packaged as a Docker image: Python 3.12 slim, dependencies installed before the code for fast rebuilds, running as a non-root user. Locally I start it with Docker Compose next to Postgres with pgvector; the API reaches the database by its service name and only starts once the database is healthy. Secrets come from a git-ignored .env, never from the image. A liveness check touches nothing, and a readiness check verifies the database and returns 503 if it is down."

Quick self-check

1 What do you build, and what do you run?

You build an image; you run a container.

2 In DATABASE_URL, why db and not localhost?

Inside Compose, localhost is the api container itself.

3 Which command deletes your database data?

docker compose down -v

4 What keeps the api from starting too early?

depends_on with condition: service_healthy

5 /health or /ready: which one checks Postgres?

/ready; /health touches nothing.

6 Where does the API key live on your laptop?

In .env, which is in .gitignore and .dockerignore.

ONE PAGE TO REVISE

Session 6 cheat sheet

Words

Dockerfile = recipe · image = kit · container = meal

Build an image, run a container

Volume = storage that survives

Dockerfile

FROM python:3.12-slim · WORKDIR /app

requirements first, then COPY app

non-root USER · CMD on ${PORT:-8000}

Image alone

`docker build -t examate-rag .`

`docker run --rm -p 8000:8000 --env-file .env examate-rag`

`docker ps` · `docker logs NAME`

Compose

services: db (pgvector:pg16) + api (build: .)

DATABASE_URL uses **db:5432**

depends_on: service_healthy · volume pgdata

Commands

`docker compose up --build` · `ps` · `logs -f api`

`stop db` / `start db`

`down` keeps data · `down -v` deletes it

Health vs ready

/health: alive, touches nothing

/ready: SELECT 1, 3 s timeout

db down → /health 200, /ready 503

Secrets

Laptop: .env (git- and docker-ignored)

CI: repo secrets (ours needs none) · Host: env settings

Leaked? Rotate the key

Remember

Ship the box, not the instructions

Inside Compose, call people by name

Health says alive, ready says can work

THREE SENTENCES FOR ANY INTERVIEW

1. "My API is a Docker image; Compose runs it next to Postgres with pgvector, and the API waits for a healthy database." 2. "Containers find each other by service name, and a named volume keeps the data." 3. "Secrets come from the environment, never from Git or the image; /ready returns 503 when the database is down."

WHERE THE FACTS COME FROM

Sources

#	PAGE	WHAT IT SUPPORTED
S1	Session doc: GenAI 03 · Deploy on day one	Compose file, /ready, vocabulary, secrets table, checkpoints, first two errors, Q1 and Q2 (code from the tested genai-week1-reference repo)
S2	Docker docs: Install Docker Desktop on Windows (docs.docker.com/desktop/setup/install/windows-install)	WSL 2 backend requirements: Windows versions, WSL 2.1.5+, 8 GB RAM, virtualization in BIOS/UEFI
S3	GitHub docs: GitHub Codespaces billing (docs.github.com/en/billing/concepts/product-billing/github-codespaces)	GitHub Free personal accounts: 120 compute hours and 15 GB-month storage per month; multiplier 2 for a 2-core machine
T	Examate template repo (github.com/govindchandak/genai-starter-repo)	Dockerfile and .dockerignore shown in section 04

ADDED FOR THIS SESSION

The image/container primer, the Dockerfile walkthrough, the "image alone" page, the no-Docker options, Q3 and errors 3–10 were written for this session. The session doc covers Compose and /ready but has no Dockerfile walkthrough.

FIVE THINGS TO REMEMBER

Recap and tonight's task

- ✓ An image is the packed box; a container is it running. Build once, run anywhere.
- ✓ Dockerfile order matters: libraries before code, so rebuilds take seconds.
- ✓ Compose runs api + db on one network; use db:5432, never localhost.
- ✓ Volumes keep data; down -v deletes it. depends_on: service_healthy orders startup.
- ✓ Secrets: .env locally, settings elsewhere, never in Git or the image. /ready checks the database.

Tonight's task: Dockerfile + docker-compose.yml working

Commit `docker-compose.yml`, the `/ready` route and `test_ready_without_database`. Add a **"Run with Docker"** section to README.md with three lines: `cp .env.example .env`, `docker compose up --build`, `curl localhost:8000/ready`. Make sure `pytest -q` passes and .env is not committed. Push before 9:45 PM. No Docker on your laptop? Use section 11 and say which option you used in PROGRESS.md.

> Your project now runs the same on **every machine**. Tomorrow it runs on the internet.

TONIGHT	BY 9:45 PM	NEXT GENAI · FRI 30 OCT
Compose + /ready pushed	Nightly GitHub check	CI and going live · week-2